



Induzione

Albero binario BTree

$e: BTree \rightarrow \mathbb{N}$ n° foglie

$h: BTree \rightarrow \mathbb{N}$ altezza

$$\begin{cases} \text{foglia} \in BTree \\ T_1, T_2 \in BTree \Rightarrow \text{branch}(T_1, T_2) \in BTree \end{cases}$$

$$\begin{cases} e(\text{foglia}) = 1 \\ e(\text{branch}(T_1, T_2)) = e(T_1) + e(T_2) \end{cases}$$

$$\begin{cases} h(\text{foglia}) = 0 \\ h(\text{branch}(T_1, T_2)) = \max(h(T_1), h(T_2)) + 1 \end{cases}$$

Dimostrare che $\forall T \in BTree. e(T) \leq 2^{h(T)}$ **Tea**

Base: $T = \text{foglia}$

$$e(T) = 1 \quad h(T) = 0$$

$$e(T) = 1 = 2^0 = 2^{h(T)} \Rightarrow e(T) \leq 2^{h(T)}$$

Passo induttivo: Ip. induttiva $T_1, T_2 \in BTree$ tali che
 $e(T_1) \leq 2^{h(T_1)} \quad e(T_2) \leq 2^{h(T_2)}$

Dobbiamo dim. che se $T = \text{branch}(T_1, T_2)$
allora $e(T) \leq 2^{h(T)}$

$$e(T) = e(\text{branch}(T_1, T_2)) = e(T_1) + e(T_2)$$

def
di e

$$\leq 2^{h(T_1)} + 2^{h(T_2)}$$

Hp. ind

$$h(T) = h(\text{branch}(T_1, T_2)) = \max(h(T_1), h(T_2)) + 1$$

prendiamo $h = \max(h(T_1), h(T_2)) \Rightarrow h \geq h(T_1)$
 $h \geq h(T_2)$

$$\leq 2^{h(T_1)} + 2^{h(T_2)} \leq 2^h + 2^h$$

$$= 2(2^h) = 2^{h+1}$$

$$= 2^{\max(h(T_1), h(T_2)) + 1}$$

$h(T)$

$$= 2^{h(T)}$$

Scoping statico e dinamico: completamente codice

```
{ int a = 0;
```

⊗

```
while (a <= 1) {
```

```
    int x; ← locale al while
```

⊗⊗

```
    x = fun();
```

```
    a++;
```

```
    {
```

```
    {
```

→ stesso valore nelle due iterazioni con scoping statico

→ valori diversi con scoping dinamico

- Per essere eseguibile dobbiamo definire `fun()` in ②
- `fun()` deve avere un ambiente non locale perché le regole di scoping abbiano effetto sul risultato
- il riferimento non locale in `fun()` deve essere definito sia nel contesto di def di `fun` che nel contesto di esecuzione di `fun`.

① → def di `fun()`
def di `x`
+ inizializzazione di `x`

② { `int x = 2;`
 `int fun()` { `return x;` }
 ↓
 per essere eseguibile
 dobbiamo inizializzare `x`

③ → inizializzazione di `x`

③ { `x = 0;`

```
{ int a=0;
  int x=2;
  int fun() { return x; }
  while (a <= 1) {
    int x;
    x=a;
    x=fun();
    a++;
  }
```

Scoping statico

La chiamata di `fun()` restituisce la `x` globale che non viene mai modificata quindi in entrambe le chiamate `x` restituisce val 2.

Scoping dinamico

La chiamata di `fun()` restituisce la `x` interna al `while` che alla prima iterazione val 2 (globale non modificata prima della chiamata).

`a++` incrementa a 1 la variabile globale `a`

⇒ alla seconda chiamata `fun()` restituisce la `x` interna al `while` che ora val 1

Scoping statico e dinamico: catena statica e CRT

```
{ int x=1; int y=2;
```

```
void A() { int z = x+y; }
```

```
void B() { int x=2; int w=4;
```

```
void C() { int x = y+w;
```

```
    A();
```

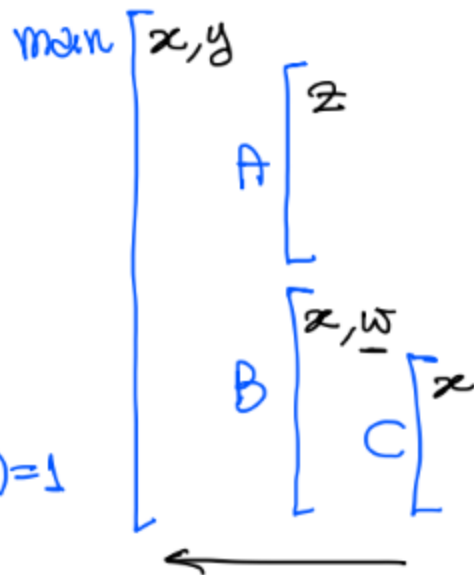
```
    y = x+y;
```

```
    C();
```

```
    B();
```

```
    }
```

$Sd(main) = 0$
 $Sd(A) = Sd(B) = 1$
 $Sd(C) = 2$



main $\rightarrow$ B $\rightarrow$ C $\rightarrow$ A sequenza di chiamate

Scoping statico

$$K = Sd(Chiamante) - Sd(Chiamato) + 1$$

il numero di volte che risolvere lungo la catena statica (sequenza di link statici) per trovare il genitore statico (RdA contesto di definizione)

CS(B)

determiniamo il link statico

$$K_B = Sd(main) - Sd(B) + 1 = 0 - 1 + 1 = 0$$

$$\Rightarrow CS(B) = main$$

Esecuzione di B : $y = x + y$

$$N_u = Sd(uso) - Sd(def)$$

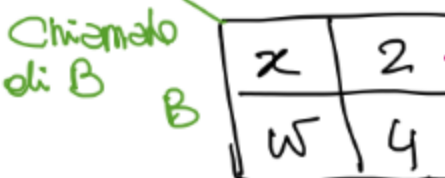
blocco in cui siamo usando x

il blocco più vicino che def x nella catena di anidamento

$$N_y = Sd(B) - Sd(main) = 1$$

$y = 2$

dice di quanto risolvere la catena statica per trovare RdA del blocco che def x



link dinamico

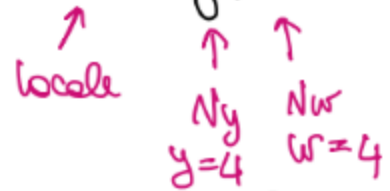
Chiamata di C



Link statico : $K_C = Sd(B) - Sd(C) + 1$
 $= 1 - 2 + 1 = 0$

Esecuzione di C $CS(C) = B$

Risoluzione : $x = y + w \rightarrow 8$



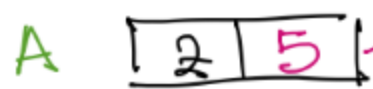
$N_y = Sd(C) - Sd(main) = 2$

Risultato della CS per trovare Rda main

$N_w = Sd(C) - Sd(B) = 1$

risultato per Rda di B

Chiamata di A



Link statico : $K_A = Sd(C) - Sd(A) + 1$
 $= 2 - 1 + 1 = 2$

$CS(A) = CS(CS(C)) = CS(B) = main$

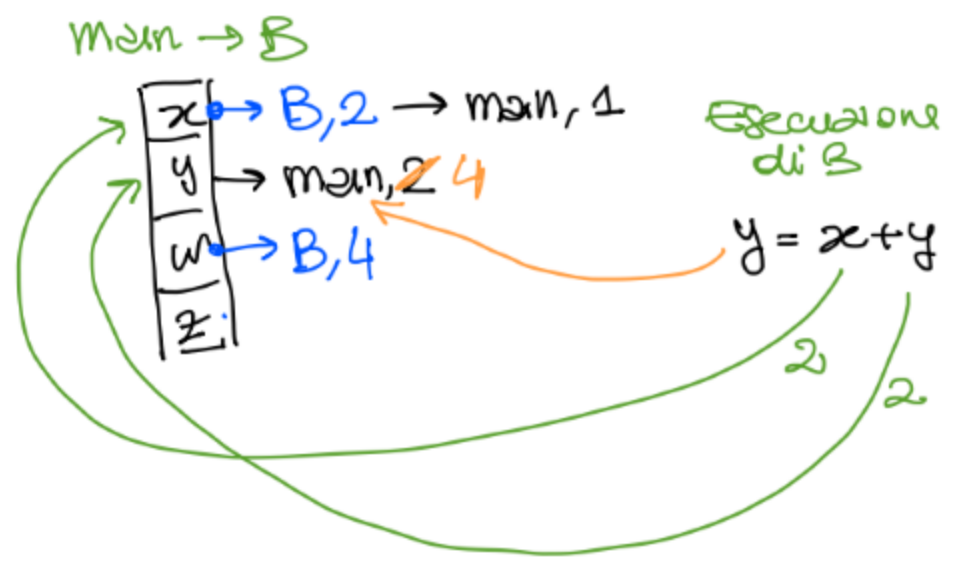
Esecuzione di A

$z = x + y$
 locale $\downarrow \downarrow$
 1 4

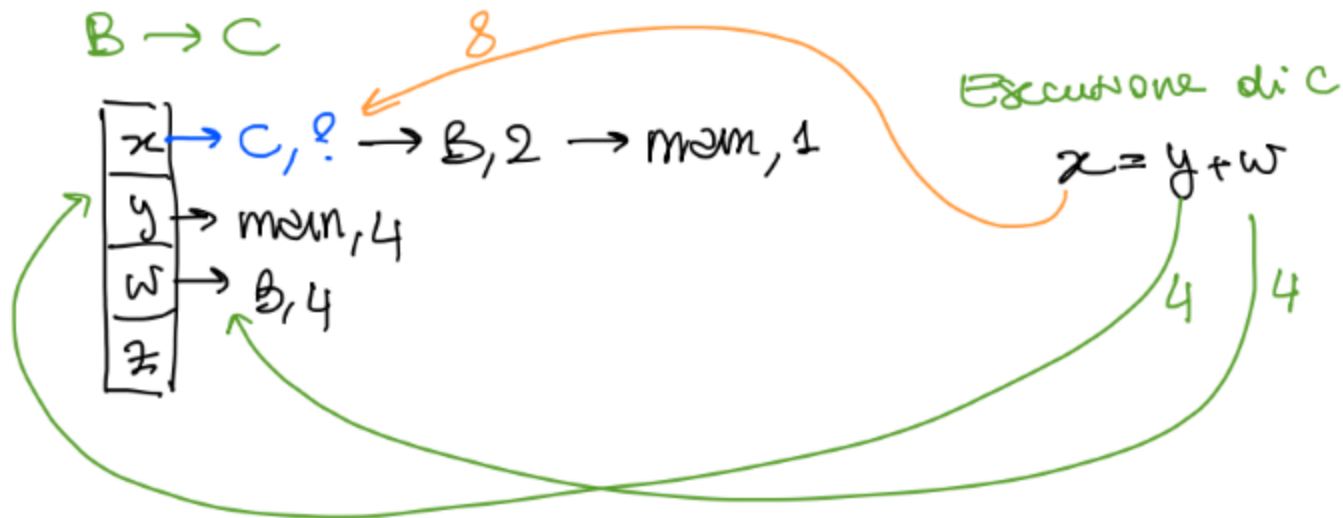
$N_x = Sd(A) - Sd(main)$
 $= N_y = 1$

Dopo la chiamata tutte le chiamate terminano e lo stack dealloca.

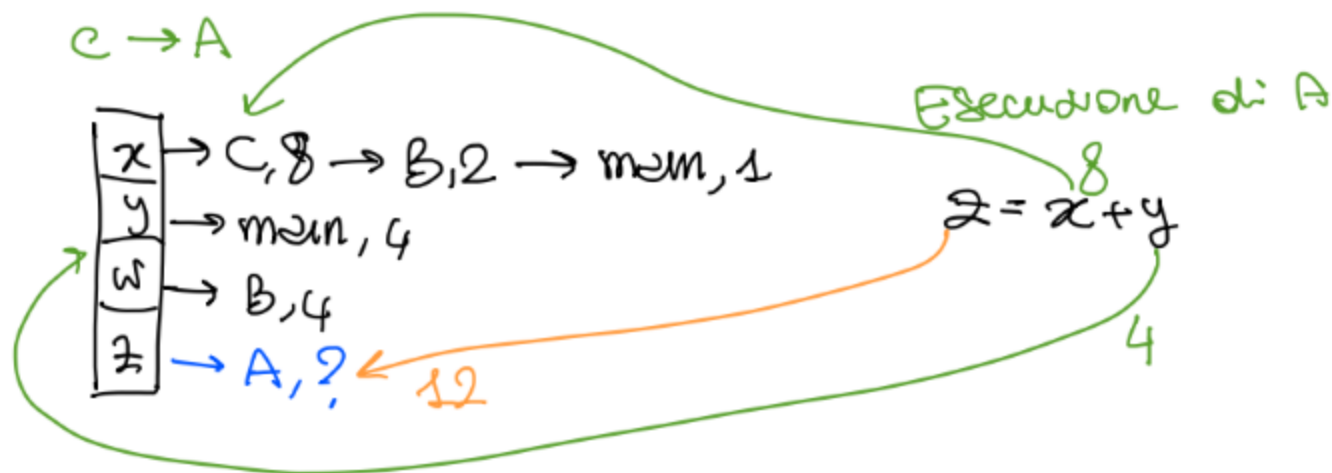
Scoping dinamico : $\rightarrow CRT$



B → C



C → A



```
int x; int y;
```

```
void fun1() { int z = x + y;
```

```
void fun2() { y = y + 1;
```

```
void fun3() { int y = x + 2;
```

```
    fun2();
```

```
    fun1();
```

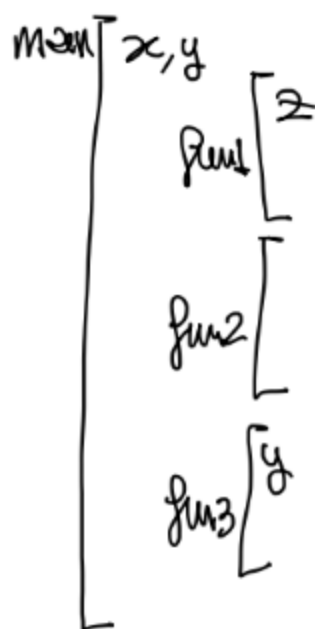
```
}
```

```
x = 5; y = 2;
```

```
fun3();
```

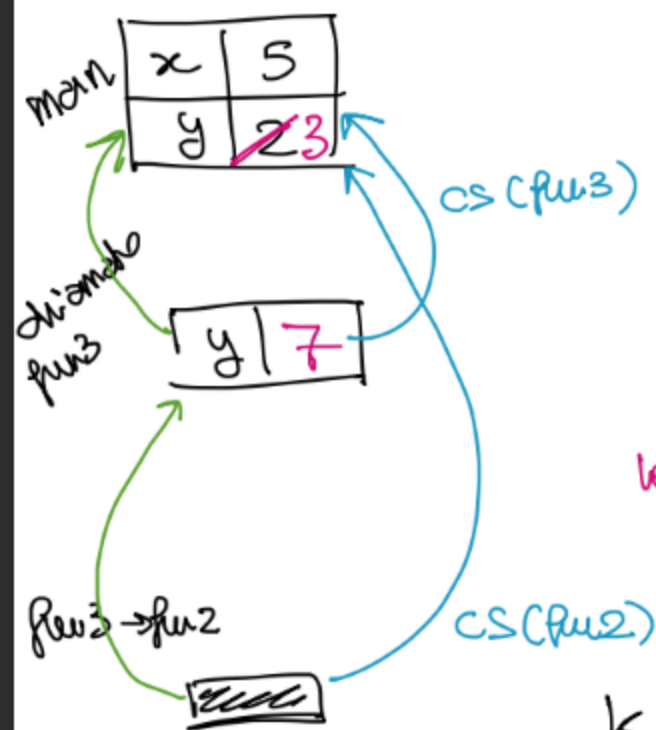
```
{
```

main → fun3() → fun2()
 ↓ fun1()



Sol(main) = 0
Sol(fun1) = Sol(fun2) = Sol(fun3) = 1

Scoping states



$x = 5$ in main
 Nx
 $y = x + 2$
 $\swarrow$
 locdb

$$K_{func3} = Sol(main) - Sol(func3) + 1 = 0$$

$$CS(func3) = main$$

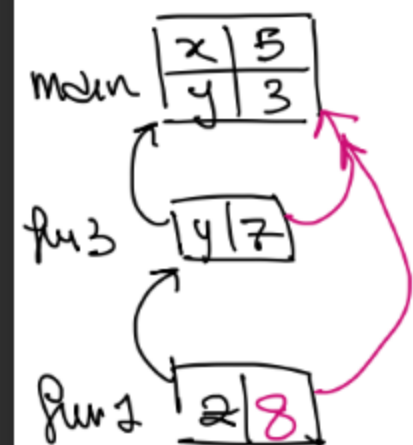
$$Nx = Sol(func3) - Sol(main) = 1$$

$$K_{func2} = Sol(func3) - Sol(func2) + 1 = 1$$

$$CS(func2) = CS(func3) = main$$

$y = y + 1$
 $\swarrow$
 3

$$Ny = Sol(func2) - Sol(main) = 1$$



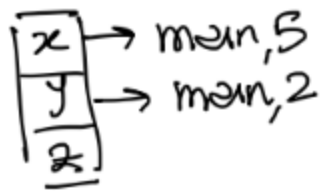
$$K_{func1} = Sol(func3) - Sol(func1) + 1 = 1$$

$$CS(func1) = CS(func3)$$

$z = x + y$
 $\swarrow$
 locdb

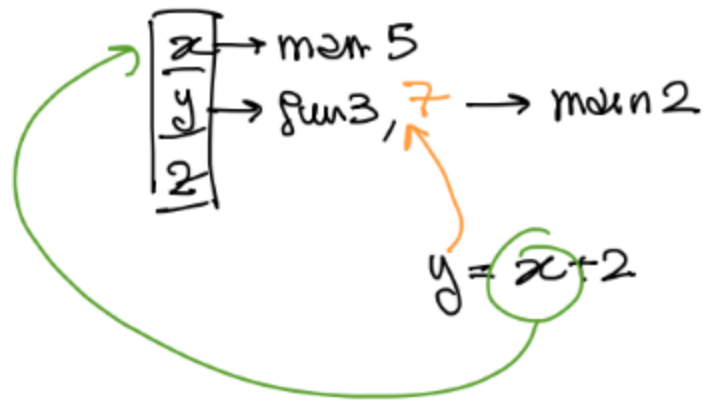
$$Nx = Ny = Sol(func1) - Sol(main) = 1$$

Scoping dinamico

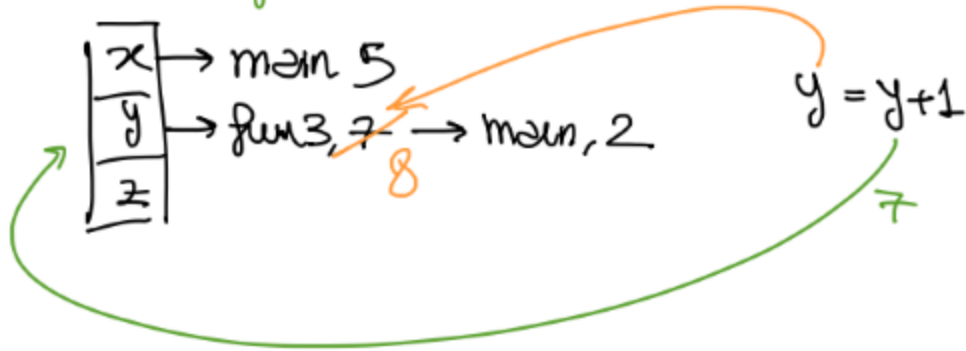


main

main → fun3



fun3 → fun2



main con
spiegoni

fun3 → fun1



Terminazione